

Agent IA: Déploiement plateforme, architecture, configuration, utilisation

Module 4

Génération augmentée par la récupération RAG)

Contenu

- 1) Principes du Retrieval-Augmented Generation (RAG) ;
- 2) Construction de bases de connaissances ;
- 3) Recherche sémantique et vectorisation ;
- 4) Connexion aux bases de données et APIs ;
- 5) Intégration souveraine

Pourquoi le RAG ?

Problématique : Les LLM ont une date de coupure de connaissances et peuvent halluciner sur des données internes ou spécifiques (ex: offres, incidents techniques).

Solution RAG : Injecter dynamiquement les avis clients ou la documentation technique pertinents directement dans le prompt du modèle.

Sécurité & Contexte : Permet de s'appuyer sur des sources vérifiées (Dataset *French Customer Review Sentiment*) sans fine-tuning lourd.

Architecture de la Recherche Sémantique

Vectorisation (Embeddings) : Transformation du texte en vecteurs numériques pour capturer le sens sémantique (et non plus seulement des mots-clés).

Chunking (Découpage) : Scinder les longs documents en morceaux cohérents pour optimiser la précision de la recherche.

Indexation : Stockage des embeddings dans une base de données vectorielle pour un calcul de distance rapide (recherche par similarité).

Infrastructure & Modèles Ouverts

Hébergement Souverain : Utilisation d'un modèle ouvert (ex: Qwen) déployé via conteneur Docker (vLLM) sur RunPod pour garantir la maîtrise des données.

Orchestration LangChain :

- ☐ Assemblage fluide du récupérateur (*Retriever*),
- ☐ Prompt enrichi
- ☐ LLM pour automatiser les réponses.

Comparatif des Bases de Données Vectorielles

Base Vectorielle	Avantages	Inconvénients	Cas d'usage idéal
Chroma DB	Très simple à installer, idéal pour le prototypage, s'intègre nativement et rapidement avec LangChain.	Moins taillé pour les environnements distribués à très grande échelle en production.	Prototypage rapide, petits/moyens volumes, environnements locaux.
FAISS (Meta)	Extrêmement rapide pour la recherche en mémoire, gère des milliards de vecteurs.	Pas de persistance native simple sans code additionnel, pas une vraie base de données.	Recherche haute performance en mémoire vive (RAM).
PGVector (PostgreSQL)	Permet de combiner données relationnelles et vecteurs dans une seule base (PostgreSQL).	Moins spécialisé pour les recherches vectorielles pures ultra-massives que les solutions dédiées.	Projets d'entreprise utilisant déjà PostgreSQL pour éviter de multiplier les technos.
Qdrant / Milvus	Robustes, orientés production, scalables, riches en fonctionnalités (filtrage métadonnées avancé).	Architecture plus complexe à déployer et administrer au quotidien.	Applications industrielles à grande échelle (Production).